



## C++ - Módulo 04

### Polimorfismo de Subtipo, Classes Abstratas e Interfaces

*Preâmbulo: Este documento contém os exercícios para o Módulo 04 dos módulos de C++.*

*Versão: 12.0*

# Sumário

|      |                                              |    |
|------|----------------------------------------------|----|
| I    | Introdução                                   | 2  |
| II   | Regras gerais                                | 3  |
| III  | Instruções de IA                             | 6  |
| IV   | Exercício 00: Polimorfismo                   | 9  |
| V    | Exercício 01: Eu não quero incendiar o mundo | 11 |
| VI   | Exercício 02: Classe Abstrata                | 13 |
| VII  | Exercício 03: Interface & recapitulação      | 14 |
| VIII | Entrega e Avaliação por Pares                | 18 |

# Capítulo I

## Introdução

*C++ é uma linguagem de programação de propósito geral criada por Bjarne Stroustrup como uma extensão da linguagem de programação C, ou "C com Classes"(fonte: [Wikipédia](#)).*

O objetivo destes módulos é apresentar a você a **Programação Orientada a Objetos**. Este será o ponto de partida de sua jornada em C++. Muitas linguagens são recomendadas para aprender OOP, mas escolhemos C++ porque é derivado de seu velho amigo C. Como C++ é uma linguagem complexa, e para manter as coisas simples, seu código estará em conformidade com o padrão C++98.

Estamos cientes de que o C++ moderno difere significativamente em muitos aspectos. Portanto, se você deseja se tornar um desenvolvedor C++ proficiente, cabe a você continuar sua jornada além do 42 Common Core!

# Capítulo II

## Regras gerais

### Compilação

- Compile seu código com `c++` e as flags `-Wall -Wextra -Werror`
- Seu código ainda deve compilar se você adicionar a flag `-std=c++98`

### Formatação e convenções de nomenclatura

- Os diretórios dos exercícios serão nomeados desta forma: `ex00`, `ex01`, `...`, `exn`
- Nomeie seus arquivos, classes, funções, funções membro e atributos conforme exigido nas diretrizes.
- Escreva os nomes das classes no formato **UpperCamelCase**. Arquivos contendo código de classe sempre serão nomeados de acordo com o nome da classe. Por exemplo:  
`NomeDaClasse.hpp`/`NomeDaClasse.h`, `NomeDaClasse.cpp`, ou `NomeDaClasse.hpp`.  
Então, se você tiver um arquivo de cabeçalho contendo a definição de uma classe "ParedeDeTijolo" representando uma parede de tijolos, seu nome será `ParedeDeTijolo.hpp`.
- A menos que especificado de outra forma, cada mensagem de saída deve terminar com um caractere de nova linha e ser exibida na saída padrão.
- *Adeus Norminette!* Nenhum estilo de codificação é imposto nos módulos C++. Você pode seguir o seu favorito. Mas lembre-se que o código que seus avaliadores não conseguem entender é um código que eles não podem avaliar. Faça o seu melhor para escrever um código limpo e legível.

### Permitido/Proibido

Você não está mais programando em C. É hora de C++! Portanto:

- Você pode usar quase tudo da biblioteca padrão. Assim, em vez de se ater ao que você já conhece, seria inteligente usar as versões em C++ das funções C que você está acostumado o máximo possível.

- No entanto, você não pode usar nenhuma outra biblioteca externa. Isso significa que bibliotecas C++11 (e formas derivadas) e **Boost** são proibidas. As seguintes funções também são proibidas: `*printf()`, `*alloc()` e `free()`. Se você usá-las, sua nota será 0 e pronto.
- Observe que, a menos que explicitamente indicado de outra forma, as palavras-chave `using namespace <ns_name>` e `friend` são proibidas. Caso contrário, sua nota será -42.
- **Você só pode usar a STL nos Módulos 08 e 09.** Isso significa: nenhum **Contêiner** (`vector`/`list`/`map`, e assim por diante) e nenhum **Algoritmo** (qualquer coisa que exija a inclusão do cabeçalho `<algorithm>`) até lá. Caso contrário, sua nota será -42.

### Alguns requisitos de design

- Vazamento de memória ocorre em C++ também. Quando você aloca memória (usando a palavra-chave `new`), você deve evitar **vazamentos de memória**.
- Do Módulo 02 ao Módulo 09, suas classes devem ser projetadas na **Forma Canônica Ortodoxa**, exceto quando explicitamente indicado de outra forma.
- Qualquer implementação de função colocada em um arquivo de cabeçalho (exceto para modelos de função) significa 0 para o exercício.
- Você deve ser capaz de usar cada um de seus cabeçalhos independentemente de outros. Portanto, eles devem incluir todas as dependências de que precisam. No entanto, você deve evitar o problema de inclusão dupla adicionando **proteções de inclusão**. Caso contrário, sua nota será 0.

### Leia-me

- Você pode adicionar alguns arquivos adicionais se precisar (ou seja, para dividir seu código). Como essas atribuições não são verificadas por um programa, sinta-se à vontade para fazê-lo, desde que você entregue os arquivos obrigatórios.
- Às vezes, as diretrizes de um exercício parecem curtas, mas os exemplos podem mostrar requisitos que não estão explicitamente escritos nas instruções.
- Leia cada módulo completamente antes de começar! Sério, faça isso.
- Por Odin, por Thor! Use seu cérebro!!!



Em relação ao Makefile para projetos C++, as mesmas regras do C se aplicam (consulte o capítulo Norma sobre o Makefile).



Você terá que implementar muitas classes. Isso pode parecer tedioso, a menos que você seja capaz de criar scripts para seu editor de texto favorito.



Você tem uma certa liberdade para completar os exercícios. No entanto, siga as regras obrigatórias e não seja preguiçoso. Você perderia muitas informações úteis! Não hesite em ler sobre conceitos teóricos.

# Capítulo III

## Instruções de IA

### ● Contexto

Este projeto foi desenvolvido para ajudá-lo a descobrir os blocos de construção fundamentais do seu treinamento em TIC.

Para ancorar adequadamente os conhecimentos e habilidades-chave, é essencial adotar uma abordagem criteriosa ao uso de ferramentas e suporte de IA.

A verdadeira aprendizagem fundamental exige um esforço intelectual genuíno — através de desafios, repetição e trocas de aprendizagem entre pares.

Para uma visão geral mais completa de nossa posição sobre a IA — como ferramenta de aprendizagem, como parte do currículo de TIC e como expectativa no mercado de trabalho — consulte as perguntas frequentes dedicadas na intranet.

### ● Mensagem principal

- ✎ Construa bases sólidas sem atalhos.
- ✎ Desenvolva verdadeiramente habilidades técnicas e de poder.
- ✎ Experimente a verdadeira aprendizagem entre pares, comece a aprender como aprender e resolver novos problemas.
- ✎ A jornada de aprendizagem é mais importante que o resultado.
- ✎ Aprenda sobre os riscos associados à IA e desenvolva práticas eficazes de controle e contramedidas para evitar armadilhas comuns.

## ● Regras para o aluno:

- Você deve aplicar o raciocínio às suas tarefas atribuídas, especialmente antes de recorrer à IA.
- Você não deve pedir respostas diretas à IA.
- Você deve aprender sobre a abordagem global da 42 em relação à IA.

## ● Resultados da fase:

Nesta fase fundamental, você obterá os seguintes resultados:

- Obter bases sólidas em tecnologia e codificação.
- Saber por que e como a IA pode ser perigosa durante esta fase.

## ● Comentários e exemplo:

- Sim, sabemos que a IA existe — e sim, ela pode resolver seus projetos. Mas você está aqui para aprender, não para provar que a IA aprendeu. Não perca seu tempo (nem o nosso) apenas para demonstrar que a IA pode resolver o problema dado.
- Aprender na 42 não é sobre saber a resposta — é sobre desenvolver a capacidade de encontrar uma. A IA lhe dá a resposta diretamente, mas isso o impede de construir seu próprio raciocínio. E o raciocínio leva tempo, esforço e envolve falhas. O caminho para o sucesso não deve ser fácil.
- Lembre-se de que durante os exames, a IA não estará disponível — sem internet, sem smartphones, etc. Você perceberá rapidamente se confiou demais na IA em seu processo de aprendizagem.
- A aprendizagem entre pares o expõe a diferentes ideias e abordagens, melhorando suas habilidades interpessoais e sua capacidade de pensar de forma divergente. Isso é muito mais valioso do que apenas conversar com um bot. Então não seja tímido — converse, faça perguntas e aprenda juntos!
- Sim, a IA fará parte do currículo — tanto como ferramenta de aprendizagem quanto como um tópico em si. Você até terá a chance de construir seu próprio software de IA. Para saber mais sobre nossa abordagem crescente, consulte a documentação disponível na intranet.



**✓ Boa prática:**

Estou travado em um novo conceito. Pergunto a alguém próximo como ele abordou isso. Conversamos por 10 minutos — e de repente, clica. Entendi.

**✗ Má prática:**

Uso secretamente a IA, copio algum código que parece certo. Durante a avaliação entre pares, não consigo explicar nada. Eu falho. Durante o exame — sem IA — estou travado novamente. Eu falho.

# Capítulo IV

## Exercício 00: Polimorfismo

|                                                                                                                       |               |
|-----------------------------------------------------------------------------------------------------------------------|---------------|
|                                      | Exercice : 00 |
| Polimorfismo                                                                                                          |               |
| Pasta de entrega : <i>ex00/</i>                                                                                       |               |
| Arquivos para entregar : <code>Makefile</code> , <code>main.cpp</code> , <code>*.cpp</code> , <code>*.{h, hpp}</code> |               |
| Funções não permitidas : Nenhuma                                                                                      |               |

Para cada exercício, você deve fornecer os **testes mais completos** que puder. Construtores e destrutores de cada classe devem exibir mensagens específicas. Não use a mesma mensagem para todas as classes.

Comece implementando uma classe base simples chamada **Animal**. Ela tem um atributo protegido:

- `std::string type`;

Implemente uma classe **Dog** que herda de **Animal**.

Implemente uma classe **Cat** que herda de **Animal**.

Essas duas classes derivadas devem definir seu campo **type** dependendo de seus nomes. Assim, o tipo do **Dog** será inicializado para "Dog", e o tipo do **Cat** será inicializado para "Cat". O tipo da classe **Animal** pode ser deixado vazio ou definido para o valor de sua escolha.

Todo animal deve ser capaz de usar a função membro:

`makeSound()`

Ela imprimirá um som apropriado (gatos não latem).

Executar este código deve imprimir os sons específicos das classes Dog e Cat, não do Animal.

```
int main()
{
    const Animal* meta = new Animal();
    const Animal* j = new Dog();
    const Animal* i = new Cat();

    std::cout << j->getType() << " " << std::endl;
    std::cout << i->getType() << " " << std::endl;
    i->makeSound(); //irá imprimir o som do gato!
    j->makeSound();
    meta->makeSound();
    ...

    return 0;
}
```

Para garantir que você entendeu como funciona, implemente uma classe **WrongCat** que herda de uma classe **WrongAnimal**. Se você substituir o Animal e o Cat pelos errados no código acima, o WrongCat deve emitir o som do WrongAnimal.

Implemente e entregue mais testes do que os fornecidos acima.

## Capítulo V

### Exercício 01: Eu não quero incendiar o mundo

|                                                                                                        |               |
|--------------------------------------------------------------------------------------------------------|---------------|
|                       | Exercice : 01 |
| Eu não quero incendiar o mundo                                                                         |               |
| Pasta de entrega : <code>ex01/</code>                                                                  |               |
| Arquivos para entregar : Arquivos do exercício anterior + <code>*.cpp</code> , <code>*.{h, hpp}</code> |               |
| Funções não permitidas : Nenhuma                                                                       |               |

Construtores e destrutores de cada classe devem exibir mensagens específicas.

Implemente uma classe **Brain**. Ela contém um array de 100 `std::string` chamado `ideas`.

Desta forma, Dog e Cat terão um atributo privado `Brain*`.

Na construção, Dog e Cat criarão seu Brain usando `new Brain()`;

Na destruição, Dog e Cat irão `delete` seu Brain.

Em sua função `main`, crie e preencha um array de objetos **Animal**. Metade dele será objetos **Dog** e a outra metade será objetos **Cat**. No final da execução do seu programa, percorra este array e delete cada Animal. Você deve deletar diretamente cães e gatos como Animals. Os destrutores apropriados devem ser chamados na ordem esperada.

Não se esqueça de verificar por **vazamentos de memória**.

Uma cópia de um Dog ou um Cat não deve ser superficial. Portanto, você tem que testar se suas cópias são cópias profundas!

```
int main()
{
    const Animal* j = new Dog();
    const Animal* i = new Cat();

    delete j; //não deve criar um vazamento
    delete i;
    ...

    return 0;
}
```

Implemente e entregue mais testes do que os fornecidos acima.

# Capítulo VI

## Exercício 02: Classe Abstrata

|                                                                                   |               |
|-----------------------------------------------------------------------------------|---------------|
|  | Exercice : 02 |
| Classe Abstrata                                                                   |               |
| Pasta de entrega : <i>ex02/</i>                                                   |               |
| Arquivos para entregar : Arquivos do exercício anterior + *.cpp, *.{h, hpp}       |               |
| Funções não permitidas : Nenhuma                                                  |               |

Criar objetos Animal não faz sentido afinal. É verdade, eles não fazem nenhum som!

Para evitar quaisquer possíveis erros, a classe Animal padrão não deve ser instanciável. Corrija a classe Animal para que ninguém possa instanciá-la. Tudo deve funcionar como antes.

Se você quiser, pode atualizar o nome da classe adicionando um prefixo A a Animal.

# Capítulo VII

## Exercício 03: Interface & recapitulação

|                                                                                   |               |
|-----------------------------------------------------------------------------------|---------------|
|  | Exercise : 03 |
| Interface & recapitulação                                                         |               |
| Pasta de entrega : <i>ex03/</i>                                                   |               |
| Arquivos para entregar : Makefile, main.cpp, *.cpp, *.{h, hpp}                    |               |
| Funções não permitidas : Nenhuma                                                  |               |

Interfaces não existem em C++98 (nem mesmo em C++20). No entanto, classes abstratas puras são comumente chamadas de interfaces. Assim, neste último exercício, vamos tentar implementar interfaces para garantir que você entenda este módulo.

Complete a definição da seguinte classe **AMateria** e implemente as funções membro necessárias.

```
class AMateria
{
    protected:
        [...]

    public:
        AMateria(std::string const & type);
        [...]

        std::string const & getType() const; //Retorna o tipo da materia

        virtual AMateria* clone() const = 0;
        virtual void use(ICharacter& target);
};
```

Implemente as classes concretas para Materias: **Ice** e **Cure**. Use seus nomes em minúsculas ("ice" para Ice, "cure" para Cure) para definir seus tipos. Claro, sua função membro `clone()` retornará uma nova instância do mesmo tipo (ou seja, se você clonar uma Materia de Ice, você obterá uma nova Materia de Ice).

A função membro `use(ICharacter&)` exibirá:

- Ice: "\* atira um raio de gelo em <nome> \*"
- Cure: "\* cura os ferimentos de <nome> \*"

<nome> é o nome do Character passado como parâmetro. Não imprima os colchetes angulares (< e >).



Ao atribuir uma Materia a outra, copiar o tipo não faz sentido.

Escreva a classe concreta **Character** que implementará a seguinte interface:

```
class ICharacter
{
    public:
        virtual ~ICharacter() {}
        virtual std::string const & getName() const = 0;
        virtual void equip(AMateria* m) = 0;
        virtual void unequip(int idx) = 0;
        virtual void use(int idx, ICharacter& target) = 0;
};
```

O **Character** possui um inventário de 4 slots, o que significa no máximo 4 Materias. O inventário está vazio na construção. Eles equipam as Materias no primeiro slot vazio que encontram, na seguinte ordem: do slot 0 ao slot 3. Se eles tentarem adicionar uma Materia a um inventário cheio, ou usar/desequipar uma Materia inexistente, nada deve acontecer (mas bugs ainda são proibidos). A função membro `unequip()` NÃO deve deletar a Materia!



Lide com as Materias que seu personagem deixa no chão como você quiser. Salve os endereços antes de chamar `unequip()`, ou qualquer outra coisa, mas não se esqueça de que você tem que evitar vazamentos de memória.

A função membro `use(int, ICharacter&)` terá que usar a Materia no slot[idx], e passar o parâmetro `target` para a função `AMateria::use`.





O inventário do seu personagem será capaz de suportar qualquer tipo de `AMateria`.

Seu **Character** deve ter um construtor que receba seu nome como parâmetro. Qualquer cópia (usando construtor de cópia ou operador de atribuição de cópia) de um **Character** deve ser **profunda**. Durante a cópia, as **Materias** de um **Character** devem ser deletadas antes que as novas sejam adicionadas ao seu inventário. Claro, as **Materias** devem ser deletadas quando um **Character** é destruído.

Escreva a classe concreta **MateriaSource** que implementará a seguinte interface:

```
class IMateriaSource
{
public:
    virtual ~IMateriaSource() {}
    virtual void learnMateria(AMateria*) = 0;
    virtual AMateria* createMateria(std::string const & type) = 0;
};
```

- **learnMateria(AMateria\*)**

Copia a **Materia** passada como parâmetro e armazena na memória para que possa ser clonada posteriormente. Como o **Character**, o **MateriaSource** pode conhecer no máximo 4 **Materias**. Elas não são necessariamente únicas.

- **createMateria(std::string const &)**

Retorna uma nova **Materia**. Esta última é uma cópia da **Materia** previamente aprendida pelo **MateriaSource** cujo tipo é igual ao passado como parâmetro. Retorna 0 se o tipo for desconhecido.

Em poucas palavras, seu **MateriaSource** deve ser capaz de aprender "templates" de **Materias** para criá-las quando necessário. Então, você será capaz de gerar uma nova **Materia** usando apenas uma string que identifica seu tipo.

Executando este código:

```
int main()
{
    IMateriaSource* src = new MateriaSource();
    src->learnMateria(new Ice());
    src->learnMateria(new Cure());

    ICharacter* me = new Character("me");

    AMateria* tmp;
    tmp = src->createMateria("ice");
    me->equip(tmp);
    tmp = src->createMateria("cure");
    me->equip(tmp);

    ICharacter* bob = new Character("bob");

    me->use(0, *bob);
    me->use(1, *bob);

    delete bob;
    delete me;
    delete src;

    return 0;
}
```

Deve exibir:

```
$> clang++ -W -Wall -Werror *.cpp
$> ./a.out | cat -e
* atira um raio de gelo em bob *$
* cura os ferimentos de bob *$
```

Como sempre, implemente e entregue mais testes do que os fornecidos acima.



Você pode passar neste módulo sem fazer o exercício 03.

## Capítulo VIII

# Entrega e Avaliação por Pares

Envie sua tarefa em seu repositório `Git` como de costume. Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Não hesite em verificar os nomes de suas pastas e arquivos para garantir que eles estão corretos.

Durante a avaliação, uma breve **modificação do projeto** pode ser ocasionalmente solicitada. Isso pode envolver uma pequena mudança de comportamento, algumas linhas de código para escrever ou reescrever, ou um recurso fácil de adicionar.

Embora esta etapa possa **não ser aplicável a todos os projetos**, você deve estar preparado para ela se for mencionada nas diretrizes de avaliação.

Esta etapa visa verificar sua compreensão real de uma parte específica do projeto. A modificação pode ser realizada em qualquer ambiente de desenvolvimento que você escolher (por exemplo, sua configuração usual), e deve ser viável em poucos minutos — a menos que um prazo específico seja definido como parte da avaliação.

Você pode, por exemplo, ser solicitado a fazer uma pequena atualização em uma função ou script, modificar uma exibição ou ajustar uma estrutura de dados para armazenar novas informações, etc.

Os detalhes (escopo, alvo, etc.) serão especificados nas **diretrizes de avaliação** e podem variar de uma avaliação para outra para o mesmo projeto.



????????????? XXXXXXXXXX = \$3\$\$6b616b91536363971573e58914295d42